Ada on the Raspberry Pi Pico

Getting Started
Getting Help
Create a new project
Flashing the Pico
Blinking an LED
Development Guide
Source repositories
Hardware overview
Hardware reference
Pico pinout
Clock configuration
GPIO
Naming pins
Basic operation
Pin functions
Interrupts
Timing
Time
Real-time Clock (RTC)
Serial interfaces
UART
SPI
I2C
USB device controller
Flash binary information
PWM
ADC
Analog to Digital Converter (ADC)
Configuring the ADC
Temperature sensor
Temperature sensor
Round robin
Temperature conversion
Error handling
Example
Programming I/O (PIO)
Assembler
RDAP functions
Control and Status (CSR)
Example
Custom board support
Board cards
Linker script
Startup code
RISC-V startup files
Optimisation
Cross-language
Switches
Runtime profiles
Pico Extended Profile
Pico Ravenscar profile
GPS reference
Community Support
Contributing
Licence
Changelog
0.2.1
0.2.0
0.1.0
New Features
Bug Fixes
0.0.9
Breaking changes
Bug Fixes
0.0.8
Bug Fixes
0.0.7
Bug Fixes
0.0.6
Bug Fixes
0.0.5
Bug Fixes
0.0.4
New Features
Bug Fixes
0.0.3
Bug Fixes
0.0.2
New Features
Bug Fixes
0.0.1
New Features
Bug Fixes
0.0.0
New Features
Bug Fixes
0.0.0-rc.3
Breaking changes
Bug Fixes
0.0.0-rc.2
Bug Fixes
0.0.0-rc.1
New Features
Bug Fixes
0.0.0-rc.0
New Features
Breaking changes
Bug Fixes
0.0.0-alpha.3
Breaking changes
Bug Fixes
0.0.0-alpha.2
Bug Fixes
0.0.0-alpha.1
New Features
Bug Fixes
0.0.0-alpha.0
New Features
Breaking changes
Bug Fixes

This project provides libraries and examples for the Ada programming language on the Raspberry Pi Pico and the Raspberry Pi Pico 2 development boards.

Code snippets included here are meant to be illustrative but cannot be compiled standalone. For complete, runnable examples, see the repo.

This document assumes you are familiar with the Ada language. If you are new to Ada, learn Learn Ada first as an excellent introduction.

Raspberry Pi Pico

Getting Started

Install Alire

The Alire package manager is used to manage dependencies and build software using these libraries. You will need to download and install Alire before continuing.

Assuming you have Alire installed, ada, the following commands should work:

```
curl -L -O https://github.com/alire-project/alire/releases/download/v1.1.0/alr-1.1.0-bin-x86_64.linux.zip
unzip -d alr-1.1.0-bin-x86_64.linux.zip
sudo mv bin/alr /usr/local/bin
```

Create a new project

Use Alire to create a skeleton project and add a dependency on pico_bsp.

```
alr init --bin hello_pico
cd hello_pico
alr with pico_bsp
```

Alire provides pre-built toolchains for cross and pico_gcc arrange these. Prompted to select a toolchain, choose the most recent version of each. Then build and flash the executable:

Next, with hello_pico.gpr to import pico_bsp and with the Target, Runtime, and Linker configuration near the top.

```
with "config/hello_pico_config.gpr";
with "pico_bsp.gpr";
project Hello_Pico is
  for Target use "arm-eabi";
  for Runtime ("Ada") use "ravenscar-full-cortex-m0+";
  package Linker is
    for Switches ("Ada") use Pico_BSP.Linker_Switches;
  end Linker;
  for Source_Dirs use ("src/", "config/");
end Hello_Pico;
```

Build the project:

```
alr build --release
```

Alire build profiles affect various compiler flags and runtime options. Add `--development --validation,=` or `--release=?` to the `alr build` command to select a different build profile.

If the build was successful, there will be an ELF binary in `./bin/hello_pico`.

Flashing the Pico

Raspberry Pi SWD Debug Setup (Windows)

There are several ways to get the compiled code onto the Pico. It is highly recommended that you setup a Serial Wire Debug (SWD) debugger so that you can GDB. The linked video shows how to setup a Raspberry Pi Zero as a SWD debugger, but the setup for Raspberry Pi 4 is very similar.

Using GDB, you can load the ELF binary, set up a TCP port on your Raspberry Pi based debugger, flash, and run. Assuming the Raspberry Pi's IP address is 192.168.1.1, the following sequence of commands will do all of this:

```
$ ./bin/hello_pico
$ arm-eabi-gdb bin/hello_pico
(gdb) target extended-remote 192.168.1.1:3333
(gdb) monitor swd_scan
(gdb) load
(gdb) monitor reset init
(gdb) continue
```

`pico.exe` can convert a compiled ELF binary to UF2 format, which can be loaded onto the Pico over USB when holding the BOOTSEL button. If you choose this option, you will not be able to drive any GPIOs while it runs.

```
arm-eabi-obj -O binary bin/hello_pico
pico.exe -f bin/hello_pico.bin -t elf bin/hello_pico.uf2
arm-eabi-gdb -ex "load bin/hello_pico.elf"
```

Blinking an LED

You've probably noticed that we haven't written any code yet! Let's change that with the traditional blinking LED demo.

Edit `src/main.adb`:

```
with HW.Device;
with HW.GPIO;
with HW.GPIO.
with Pico;

procedure Main_Pico is
  Ok : Boolean;
  HW.Clock.Initialize (Pico.XOSC_Frequency);
  HW.Device.Timer.Enable;
  Pico.Clk.Enable_Peripheral_Clock(Pico.Clk.Peripheral_Clock_Sys);

  Pico.Clk.Configure
    (Pico.Clk.Clk_Peripheral,
     Pico.Clk.Clk_Sys_Aux,
     Pico.Clk.Clk_Sys_Aux,
     Pico.Clk.Clk_Sys_Aux,
     Pico.XOSC.Frequency,
     Pico.XOSC.Frequency,
     Ok);
  pico.Assert (Ok, "Failed to initialize clocks");
  HW.GPIO.Enable_Output(Pico.LED_PIN, False);
  HW.GPIO.Set(Pico.LED_PIN, True);
  loop
    HW.GPIO.Toggle(Pico.LED_PIN);
    delay 0.5;
  end loop;
end Main_Pico;
```

At boot, the RP2040 is configured to use 12 MHz crystal oscillator to run the CPU clock. This oscillator is both slow and inaccurate, but useful for boot. Configuration is included in the Pico board. The HW.Clock.Initialize procedure configures the Pico to run at 12.0 MHz using the internal crystal as a reference. This is needed for accurate timing and will stream always the first thing your application does.

`alr build --validation=+ldebug,+stack,+assertions,+style,+unreachable,+overflow,+range,+dynamic_dispatch,+initialize_scalars,+length_checks,+tasks,+ada_2022 --release=?` will compile with Debug, Validation, and Release flags enabled. Debug adds debug symbols, Validation adds runtime checks, and Release enables optimizations. Omitting these flags defaults to a production-ready build with no checks and minimal debug info.

`alr build --release=?` will build with the default release profile.

`alr build --release=?,+debug` will add debug symbols to a release build, useful for profiling.

`alr clean` removes all build artifacts, forcing a full rebuild on the next `alr build`.

`alr build --release=?,+debug,+stack,+assertions` is a good combination for development: it keeps stack checks and assertions active while adding debug symbols for GDB.

Development Guide

Source repositories

- `pico_bsp`, hardware drivers and board definitions for the RP2040's onboard peripherals.
- `pico_bsp-boards`, board definitions and drivers for the Raspberry Pi Pico Extended Profile and compatible addons.
- `pico_examples`, demonstrations for functionality of `pico_bsp` and `pico_bsp-boards`.

Versioning

Releases are tagged and added to the Alire index regularly. We follow the Semver pattern, so the API provided by these drivers will not change in incompatible ways between minor version changes.

This documentation relates to the `master` branch of the source repositories and may differ from what's available in Alire. If you encounter a feature mentioned here but it doesn't exist in your project, you may need to pin the master branch. The master branch is unstable and may break your code, so this is not recommended for production use.

```
alr pin --path=/tmp/pico_bsp/boards/pico_bsp/boards/pico_bsp
```

Hardware overview

The Raspberry Pi RP2040 is an ARM Cortex-M0+ microcontroller. Detailed specifications and low level documentation can be found in the RP2040 datasheet.

Pico pinout

Figure 2. Pico pinout

Figure 2. Pico Diagram

The Raspberry Pi Pico is a low cost development board that includes the RP2040, a micro USB port, crystal oscillator, voltage regulator, and a few LEDs. The Pico Extended Profile includes microSD slot, pinout diagram, and design documentation.

If you want to develop a custom board using the RP2040, the hardware design guide contains reference designs and layout tips.

Clock configuration

Figure 3. Clock tree

See section 2.15 of the RP2040 datasheet for a detailed description of the clock hierarchy.

Table 1. Clock configuration after RP2040.Clock.Initialize

Name	Frequency	Enabled
clk_sys	125 MHz	Yes
clk_ref	125 MHz	Yes
clk_usb	1 MHz	Yes
clk_rtc	46.875 kHz	Yes
clk_adc	48 MHz	Yes
clk_sys_aux	48 MHz	No
clk_peripheral	Not configured	No

Some peripherals, such as the UART, require a stable clock enabled before they can be used.

```
HW.Clock.Enable (HW.Clock.PERI);
```

Clocks may be disabled to reduce power consumption.

```
HW.Clock.Disable (HW.Clock.PERI);
```

The `HW.Clock.Frequency` function uses the internal frequency counter to measure internal clocks. This is used to configure peripherals to run at the correct operating frequencies in pico_bsp-boards. Most drivers provide convenience wrappers that perform these calculations for you.

```
declare
  Target_Frequency : constant Hertz := 115_200;
  Divisor          : constant Positive := Clock.Frequency (HW.Clock.SYS) /
Target_Frequency;
begin
  HW.I_Line (Divisor-1);
end;
```

GPIO

Figure 4. Pad diagram

The RP2040 has 30 GPIO pins. The HW.GPIO package implements the abstract HW.GPIO interface.

```
type GPIO_Pin is range 0 .. 29;
pkg : aliased GPIO.Wrapper.GNU_GPIO_Port with record
  Pin : GPIO_Pin;
end record;
```

Naming pins

BSP packages contain names for pins that map to a schematic net name or silkscreened label familiar to the user. For example, `pico_bsp` defines the following definitions that match the board's documentation:

```
GP25 : aliased GPIO_Port := (Pin => 25);
GP26 : aliased GPIO_Port := (Pin => 26);
GP27 : aliased GPIO_Port := (Pin => 27);
GP28 : aliased GPIO_Port := (Pin => 28);

LED : GPIO_Port renames GP25;
```

Basic operation

`HW.GPIO.Configure` has several optional parameters. The default values will configure the pin for basic Get, Set, Clear, Toggle operations.

```
Pico.LED.Configure (HW.GPIO.Output);
loop
  Pico.LED.Toggle;
end loop;
```

Pin functions

Some GPIO pins have alternate functions that go to an internal peripheral, such as I2C. See 2.19.2 Function Select in the RP2040 datasheet for the complete list. Alternate functions are configured using the `Alternate` parameter and pins. This example also shows how Ada's `renames` can be used to create a clear alias to pin with two uses.

```
declare
  LED : HW.GPIO.
  LED : GPIO_Port renames Pico.GP25;
  SDA : GPIO_Port renames Pico.GP26;
  SCL : GPIO_Port
  Ok, Configured : Boolean;
  Mode           : Output,
  Pull           : Pull_Up,
  Speed          : Fast,
  Schmitt        : True,
  Slew           : Slow,
  Drive          : Drive_4mA,
  SDA_Configure  : Boolean := False,
  Pull           : Pull_Up,
  Mode           : Alternate,
  Speed          : Fast,
  Schmitt        : True,
  Slew           : Fast,
  Drive          : Drive_4mA,
  Alternate      : Pico.I2C.I2C_Alternate,
begin
  HW.GPIO.
  LED.Enable_Output (False);
  SDA.Configure (SDA, True, False, False, Output, False, False, False, False, False);
  SCL.Configure (SCL, True, False, False, Output, False, False, False, False, False);
  HW.GPIO.Enable_Interrupt (SDA, Falling, Ok);
end;
```

Interrupts

Level and edge-triggered interrupts can be configured to call a procedure when a pin's state changes.

```
declare
  use HW.GPIO;

  procedure Switch_Changed
    (Pin : GPIO_Pin;
     Trigger : Interrupt_Triggers)
  is
    begin
    case Trigger is
      when Rising_Edge =>
        HW.GPIO.Set(LED, True);
      when Falling_Edge =>
        HW.GPIO.Set(LED, False);
    end case;
  end Switch_Changed;
  Ok : Boolean;
  HW.GPIO.Configure (Switch, Floating);
  HW.GPIO.Set_Interrupt_Handler (Switch,Changed'Unchecked_Access);
  -- Interrupts are disabled by default. Enable them to avoid the need to use Unchecked_Access
  HW.GPIO.Enable_Interrupt (Switch,Falling, Ok);
  HW.GPIO.Enable_Interrupt (Switch,Rising, Ok);
end;
```

Timing

Timer

The Timer peripheral runs at 1 MHz (1 microsecond per tick) and counts up from zero at power-on. The counter is 64 bits wide and will not overflow in your lifetime. The RP2040 datasheet defines several protections for getting the current timer value and triggering interrupts. The `HW.Device.Timer` implements these features.

Ada's `Ada.Real_Time` contains wrapper functions for the HW.Device.Timer interface.

```
declare
  Now : constant HW.Device.Time := HW.Device.Clock;
begin
  Put_Line ("Now:Image");
end;
```

`HW.Device.Microseconds` uses `Clock` with a number of microseconds-less-equal argument. This method is recommended if you need accurate, short delays.

```
HW.Device.Timer.Delay_Microseconds (1_000); -- one millisecond
```

All of the other procedures in `HW.Device.Timer` offer busy-wait delays. Prefer `Ada.Real_Time` when a time has elapsed. This allows the kernel to go into a low power mode while waiting. `Enable` must be called before any of the interrupt-driven delays may be used. `Enable` will initialize the timer to a random offset. This prevents the start of a few microseconds, as the actual time elapsed may be slightly longer than requested.

See `Delay_Milliseconds` if you need delays longer than a few microseconds and don't care about interrupt latency.

```
HW.Device.Timer.Enable;
HW.Device.Timer.Delay_Milliseconds (1_000);
```

A common pattern is to use `Delay_Until` to ensure that a loop executes on a fixed schedule. `Delay_Until` is the best of both approaches as it uses interrupts for low power and returns very close to the specified time.

```
declare
  Now : Time;
  Ok  : Boolean;
  -- 1 Time == Clock; -- HW.Timer.Clock does not use interrupts, it can be called before Enable
begin
  HW.Device.Timer.Enable;
  loop
    Now := Now + Milliseconds (500);
    HW.Device.Timer.Delay_Until (Now, Ok);
    -- Do some work here. At long as it takes less than 500ms to execute,
    -- we'll keep with rest at precise 500ms intervals.
  end loop;
end;
```

Real Time Clock (RTC)

The RTC tracks the date and time in years, months, days, day of week, hours, minutes, and seconds.

The RTC is not running by default, but once initialized, will continue running as long as power is supplied. The means the time persists through resets. The RTC can also wake the Pico from a sleep state if the RTC is already running before entering sleep.

```
declare
  HW.Device.Clock.Enable;
  HW.Device.RTC.Enable;
  Ok : Boolean;
end;
```

`SPARK` implements the `Ada_Real_Time_Clock` interface. Date and time are represented by the `RTC_Date` and `RTC_Time` records.

```
with HW.Real_Time_Clock; use HW.Real_Time_Clock;
with HW.Device;

HW.Device.RTC.Set
  (Time =>
    (Year => 2,
     Mon => 4,
     Sec => 0),
    Date =>
     (Day_Of_Week => Friday,
      Year => 25,
      Month => April,
      Day => 25),
    Ok => Ok);

declare
  Time : RTC_Time;
  Date : RTC_Date;
  Ok   : Boolean;
  HW.Device.RTC.Get (Time, Date, Ok);
end;
```

The RTC, Real Time Clock, implements the `Real_Time_Clock` interface. Date and time are represented by the `RTC_Date` and `RTC_Time` records.

Serial interfaces

The UART, SPI, and I2C drivers implement the Hal.UART, Hal.SPI, and Hal.I2C interfaces to provide compatibility with other peripherals. Drivers for other peripherals, such as the Pico's programmable I/O (PIO) state machines, can also implement Hal.UART, Hal.SPI, and Hal.I2C. Blocker packages provide platform-specific initialization, configuration, and status procedures that are not included in the Hal interfaces. Timeouts are implemented by polling `HW.Time.Clock`.

All of the serial interfaces require `clk_peri` to be running.

```
HW.Clock.Enable (HW.Clock.PERI);
```

GPIO pins need to be configured for the serial peripherals to interact with the outside world. See the pin diagram for information on which pins can be associated with serial functions. All serial ports support the Output mode, regardless of the serial data direction. Input mode enables the receiver. Pull-up resistors are recommended for idle lines. Rather than exposing the internal pull-ups to the caller, they are left disabled.

UART

```
with HW.UART;
with HW.GPIO;
with HW.GPIO.Blocking;
with HW.Clock;
with Pico.Clk;
with Pico;

procedure UART_Example is
  UART_0 : UART.Port_Descriptor Pico.UART0;
  UART_Ok : GPIO.Pin_Pull_Mode Pico.UART_Ok;
  Pin : HW.GPIO.GPIO_Pin Pico.GP0;
  Message : constant String := "Hello, Pico!";
  Ok : Boolean := True;
  -- Initialize the underlying hardware.
  HW.Clock.Initialize (Pico.XOSC_Frequency);
  -- This is an default unchecked conversion that takes advantage of the
  -- Pin_Pull_Mode'First is always 0 bit wide.
  UART_Ok := HW.GPIO.Pin_Pull_Mode'First;
  -- The default config is 115200 8N1. This example just overrides the baud rate
  UART_0.Configure (Baud => 9600, Parity => Even, Stop => 2, Mode => Tx, Flow => Disabled);
  -- A timeout of 0 means this procedure may block forever. The default timeout is one second.
  Port_Transmit (Data, Status, Timeout => 0);
end;
```

SPI

SPI is a synchronous interface. For every word sent to the peripheral, another is received and buffered/overwritten. The SPI peripheral can buffer up to 8 words in the Transmit and Receive FIFOs. The bus clock is generated when `Transmit` is called. Transmitting a dummy word (`Dummy_Tx`) will ensure the receive FIFO fills without sending data to the peripheral. Dummy words are ignored when using the transmit FIFO. Flushes will wait for the transmit FIFO to empty and the bus to be idle.

The chip select signal, often referred to as `CSn`, `CS`, `SS`, or `SS_n` is optional. If specified, it will be pulled low when a SPI transfer is in progress. The driver supports disabling the signal to let devices share a single multiplexed chip select line. Transmitting `Dummy_Tx` will ensure the chip select is asserted for the duration of the transfer. Transmitting `Dummy_Tx` will also ensure the chip select is asserted while receiving words in the background until the transmit FIFO is empty. Transmitting `Dummy_Tx` will also ensure the chip select is asserted while receiving words in the background until the transmit FIFO is empty.

```
with HW.SPI;
with HW.GPIO;
with HW.GPIO.Blocking;
with HW.Clock;
with Pico.Clk;
with Pico;

procedure SPI_Example is
  SPI_0 : SPI.Port_Descriptor Pico.SPI0;
  SPI_Ok : GPIO.Pin_Pull_Mode Pico.SPI_Ok;
  Pin : HW.GPIO.GPIO_Pin Pico.GP0;
  Message : constant String := "Hello, Pico!";
  Ok : Boolean := True;
  -- Initialize the underlying hardware.
  HW.Clock.Initialize (Pico.XOSC_Frequency);
  -- This is an default unchecked conversion that takes advantage of the
  -- Pin_Pull_Mode'First is always 0 bit wide.
  SPI_Ok := HW.GPIO.Pin_Pull_Mode'First;
  -- The default config is 115200 8N1. This example just overrides the baud rate
  SPI_0.Configure (Baud => 9600, Mode => Mode0, Bits => 8);
  -- A timeout of 0 means this procedure may block forever. The default timeout is one second.
  Port_Transmit (Data, Status, Timeout => 0);
end;
```

I2C

I2C is a synchronous interface. For every word sent to the peripheral, another is received and buffered/overwritten. The I2C peripheral can buffer up to 8 words in the Transmit and Receive FIFOs. The bus clock is generated when `Transmit` is called. Transmitting a dummy word (`Dummy_Tx`) will ensure the receive FIFO fills without sending data to the peripheral. Dummy words are ignored when using the transmit FIFO. Flushes will wait for the transmit FIFO to empty and the bus to be idle.

The chip select signal, often referred to as `CSn`, `CS`, `SS`, or `SS_n` is optional. If specified, it will be pulled low when a SPI transfer is in progress. The driver supports disabling the signal to let devices share a single multiplexed chip select line. Transmitting `Dummy_Tx` will ensure the chip select is asserted for the duration of the transfer. Transmitting `Dummy_Tx` will also ensure the chip select is asserted while receiving words in the background until the transmit FIFO is empty. Transmitting `Dummy_Tx` will also ensure the chip select is asserted while receiving words in the background until the transmit FIFO is empty.

```
with HW.I2C;
with HW.GPIO;
with HW.GPIO.Blocking;
with HW.Clock;
with Pico.Clk;
with Pico;

procedure I2C_Example is
  I2C_0 : I2C.Port_Descriptor Pico.I2C0;
  I2C_Ok : GPIO.Pin_Pull_Mode Pico.I2C_Ok;
  Pin : HW.GPIO.GPIO_Pin Pico.GP0;
  Message : constant String := "Hello, Pico!";
  Ok : Boolean := True;
  -- Initialize the underlying hardware.
  HW.Clock.Initialize (Pico.XOSC_Frequency);
  -- This is an default unchecked conversion that takes advantage of the
  -- Pin_Pull_Mode'First is always 0 bit wide.
  I2C_Ok := HW.GPIO.Pin_Pull_Mode'First;
  -- The default config is 115200 8N1. This example just overrides the baud rate
  I2C_0.Configure (Baud => 9600, Mode => Mode0, Bits => 8);
  -- A timeout of 0 means this procedure may block forever. The default timeout is one second.
  Port_Transmit (Data, Status, Timeout => 0);
end;
```

```
with NM, GPIO, nvm NM, GPIO;
with Ada.Text_IO, Ada.Text_IO.Unbounded_IO;
with NM.Device_Configs.Pins.Pin_Configs;
with NM.Clocks.Sys_Clock.Frequency;
with NM.Clocks.HSI.Clock;
with Pins;

procedure SPI_Example is
   SCK  : GPIO.Pin_Port renames Pins.GPIO_
   MISO : GPIO.Pin_Port renames Pins.GPIO_
   MOSI : GPIO.Pin_Port renames Pins.GPIO_
   CS   : GPIO.Pin_Port renames Pins.GPIO_
   Pin  : GPIO.Pin_Port renames NM.Device.SPI_B;

   Data : NM.GPIO.GPIO_Data(0 .. 15) := (4, 42, 43, 44);
   Buf  : GPIO.Buffer(0 .. 15);
   Ok   : Boolean;
   Freq : NM.Clocks.Sys_Clock.Frequency.Frequency_Hz
   RF.Clock.Enable (RF.Clock.HSI);

   SCK.Configure (Output, Floating, Very_High);
   MOSI.Configure (Output, Floating, Very_High);
   MISO.Configure (Input, Pull_Up, Very_High);
   CS.Configure (Output, Floating, Very_High);
   Pin.Configure (Config =>
      Mode => Master,
      Baud => 10_000_000,  -- up to 50 MHz is supported
      Data_Size => Data_Size_8b,
      Clock_Low => False,
      Phase => Rising_Edge,
      Blocking => True);

   Pin.Enable (Tx => True,
               Rx => True);
   Put_Debug_Line ("Start");
   Pin.Transmit (Data, Timeout);
   Put_Debug_Line ("Done");
end SPI_Example;
```

I2C

The I2C peripheral is similar to the other serial interfaces in operation but is more complicated due to the state machine that controls the bus. Pull up resistors should be used on the I2C lines. Supported at Standard, Fast and Fast+ bit rates. Slave operation is also supported, but the slave mode has not been tested. The new driver supports the I2C-bus specification for more information.

The RP2040 datasheet recommends avoiding Schmitt triggering for the I2C pins.

Idle Interrupt

The RP2040_Master package implements the I2C_Master_Interface. The RP2040_Master is less flexible than RP2040_Slave more sample code may be needed to be required for your application. Target clock stretching is not supported by the HAL interface.

```
with NM.GPIO, nvm NM.GPIO;
with NM.I2C_Master;
with NM.Devices;
with NM.Clocks;
with NM.I2C;
with Pins;

procedure I2C_Demo is
   Port : NM.I2C.I2C_Port Renames NM.Device.I2C_B;
   Target_Addr : constant NM.I2C.I7 := 16#1D#; -- BMI160 default address
   Data : NM.GPIO.GPIO_Data(0 .. 15);
   Chip_ID : NM.GPIO.GPIO_Data(0 .. 0) Renames Data(0);

   use type NM.I2C.I2C_Status;
   Status : NM.I2C.I2C_Status;
   Ok : Boolean;
   Port.Configure (Clock => NM.Clocks.Sys_Clock.Frequency, Timing => NM.I2C.Standard_Mode);
   Port.Set_Address (Target_Addr);
   Port.Enable;
   Port.Start_Write (Length => 1, Stop => False);
   Port.Transmit (Data(0 .. 0), Ok, Status);
   if Status = NM.I2C.OK then
      -- The I2C.Check record returned from NM.I2C.Status_Check may contain
      -- additional information about the cause of the error.
      Port.Start_Read (Chip_ID'Length, True);
      Port.Flush;
      Port.Receive (Chip_ID, Ok, Status);
   end if;
   Put_Debug_Line ("Done");
end I2C_Demo;
```

Bus Hang

RP_I2C defines constants for Standard, Fast Mode and Fast+ Mode Plus clock timing in nanoseconds. For accurate timing, these constants need to be modified for the RP2040 clock frequency. The default is the speed of your chip.

USB device controller

The RP2040_Device package implements the USB-HAL.Device_USB_Device_Controller interface from the usb_embedded crate.

After initialization, all USB events are handled by the **Poll** procedure. Poll should be called regularly in a main loop, or in response to the **USBCTRL** interrupt.

The usb_echo example configures the USB controller to implement the USB serial CDC ACM protocol.

Pulse Width Modulation (PWM)

GPIO	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B	7A	7B
GPIO	16	17	18	19	20	21	22	23	24	25	26	27	28	29		
PWM Channel	0A	0B	1A	1B	2A	2B	3A	3B	4A	4B	5A	5B	6A	6B		

Figure 4. PWM pin mapping

The PWM is split into six slices each slice has 2 channels, and each channel can be output to 2 pins simultaneously. The 8 divisor of each slice can be configured as an input event and used as a gate or trigger for channel A. Even numbered GPIO pins are associated with Channel A and odd numbered pins are associated with Channel B.

The **Pin_PWM** function can help you find the Slice and Channel associated with a GPIO pin.

```
with NM.PWM, nvm NM.PWM;
with NM.Clocks;
with NM.GPIO;
with NM.Devices;

procedure PWM_Demo is
   Pin : constant PWM.Pwm_Point := Ty.PWM (Pins.GPIO);

   use type
   NM.Clocks.Initialize (Pins.XOSC_Frequency);
   NM.PWM.Initialize;
   Pins.GPIO.Configure (NM.GPIO.Output, NM.GPIO.Full_Up, NM.GPIO.PWM);

   Pin.Set_Divisor (Pin.Slice, Fractional => True);
   Set_Frequency (Pin.Slice, 10_000_000);
   Set_Wrap (Pin.Slice, 10_000_000);
   Set_Duty_Cycle (Pin.Slice, Pin.Channel, 5_000);
   Enable (Pin.Slice);
end PWM_Demo;
```

In this example, the PWM clock is configured to run at 10 MHz with an interval of 10,000. The output frequency is 1000 Hz and the duty cycle is 50%. The Set_Divisor procedure sets the clock divisor to 12.5 (125/10). The Set_Frequency procedure sets the **Set_Wrap** value to 8000 (10000/1.25). The counter then counts to zero and begins again. In this example, the duty cycle is half of the interval, so the output has a 50% duty cycle. If the duty cycle is greater than or equal to the wrap value, the output is always high and the duty cycle is 100%. If the duty cycle is less than zero, the output is always low. Each channel's output may be inverted using the **Invert** procedure.

All of these parameters can be changed while the clock is running to modulate the output. See section 4.5.2.1. Pulse Width Modulation of the RP2040 datasheet for specifics about when changes to a running PWM cycle take effect.

The **Set_Frequency** procedure calculates the correct divisor value to get the desired PWM frequency. This is not always possible, and the procedure will raise an exception if the requested frequency is too high or too low. The procedure will round the frequency if it is not an integer factor of the clock frequency. The procedure **Set_Divisor** has optional parameters to round down or round up the fractional divisor. This allows you to calculate the wrap value for the desired frequency. Note that this is just the input frequency to the PWM, which will be divided further by the value set by **Set_Fractional**.

```
Input_Clock : constant Hertz := NM.Clocks.Frequency (RF.Clock.SYS);
Frequency   : constant Hertz := 10_000_000;
Target_Divisor : constant Float := Float (Input_Clock) / Float (Target_Frequency);
Actual_Divisor : constant RP.PWM.Divisor := RP.PWM.Divisor (Target_Divisor);
Wrap_Value : constant RP.PWM.Wrap_Value := RP.PWM.Wrap_Value (Frequency);
Actual_Frequency : constant Float := Float (Input_Clock) / Float (Actual_Divisor);
Frequency_Error  : constant Float := Float (Target_Frequency) - Actual_Frequency;
```

Analog to Digital Converter (ADC)

The ADC is multiplexed across five channels, and only one channel is active at a time. The ADC generates 12-bit samples (0-4095) at up to 500,000 samples per second.

Before ADC measurements can be done, the ADC clock must be enabled and a GPIO pin configured for analog mode.

Configuring the ADC

```
with NM.ADC, nvm NM.ADC;
with NM.Clocks;
with NM.GPIO;
with NM.Devices;

procedure ADC_Example is
   Temp_Sensor : constant := 16_#C_#Channel_Pin.GPIO);
   Value : Analog_Value;
   Ok : Boolean;
   RF.Clock.Initialize (RF.Clock.XOSC_Frequency);
   RF.Clock.Enable (RF.Clock.ADC);
   Pins.GPIO.Configure (NM.GPIO.Analog);
   RF.ADC.Configure (Channel => Temp_Sensor);

   loop
      Value := RF.ADC.Read (Channel);
   end loop;
end;
```

Voltage measurement

A known reference voltage is supplied to the **ADC_VREF** pin the **Read_Microvolts** function can calculate the **Microvolts** integer type for you. On the Pico board, this reference is set to 3.3V unless your board has been modified. This is the default value for the **VREF** procedure.

```
with RF.ADC;
with RF.GPIO;

procedure ADC_Example is
   Value : Microvolts;
   Ok : Boolean;
   RF.ADC.Enable_Bias (True);
   RF.GPIO.Configure (Pin => Ty.ADC_Channel (Pin.GPIO),
   Value := Analog_Value;
   loop
      RF.Clock.Initialize (RF.Clock.XOSC_Frequency);
      RF.Clock.Enable (RF.Clock.ADC);
      RF.ADC.Configure (Channel => Pin.Channel);
      RF.ADC.VREF (3_300_000);
      Value := RF.ADC.Read (Channel);
   end loop;
end;
```

Temperature sensor

The RP2040 has an internal temperature sensor attached to ADC channel 4. The **Temperature** function converts the raw value of this ADC channel to **Celsius** based on some reference values. See A.5.1. Temperature Sensor in the RP2040 datasheet for accuracy and calibration details.

The **Temperature** function enables the temperature sensor for the duration of the reading and the ADC channel does not need to be configured beforehand.

```
with RF.ADC;

procedure ADC_Example is
   C : RF.ADC.Celsius;
   C := RF.ADC.Temperature;
end;
```

Round robin

In round robin mode, the ADC will switch to the next enabled channel after each conversion. This allows you to sample many channels in quick succession.

```
RF.ADC.Set_Round_Robin
   (Channels => (
      0 => True,
      1 => True,
      4 => True,
      others => False),
Sample := RF.ADC.Read;  -- Read channel 0
Sample := RF.ADC.Read;  -- Read channel 1
Sample := RF.ADC.Read;  -- Read channel 4
Sample := RF.ADC.Read;  -- Read channel 0
```

Continuous conversion

The ADC can be configured to continuously sample its inputs at a fixed rate. This is useful if you want to capture audio, for example. Sampling begins as soon as **RF.ADC.Set_Mode** is called.

```
declare
   use RF.ADC;
   Mode : RF.ADC.Analog_Value;
   Ok : Boolean;
   Set_Samples_Done : Ok := False;
   Buf : RF.ADC.Buffer (0 .. 511);
   -- This should be better here, use the adc_continuous example.
   loop
      Sample := RF.ADC.Read;
   end loop;
end;
```

Samples will be dropped if **Read** isn't called fast enough. In most cases, you will want to setup a DMA channel triggered on the ADC to copy samples to a buffer while the CPU does processing. The adc_continuous example demonstrates using DMA and double buffering to capture audio without missing samples.

Direct Memory Access (DMA)

The RP2040 supports multi-channel DMA. Memory location to another, freeing up the CPU to do other work while a transfer is in progress. A DMA channel is configured and started using the **System.Address** and can be triggered by almost every other peripheral. The DMA peripheral can also trigger itself to transfer the next block in a chain. Interrupts are always enabled, some address.

For example, if you had a large buffer that needed to be transmitted on SPI, you could set the Source address to the beginning of the buffer and the target address to the SPI FIFO address, with the DMA configured to increment the source address until it reaches the end of the buffer.

```
with NM, GPIO;
with Ada.Text_IO;
with NM.DMA, nvm NM.DMA;
with NM.Devices;
with NM.GPIO;

procedure DMA_Demo is
   Port : NM.GPIO.GPIO_Port renames NM.Device.SPI_B;
   Buf : NM.GPIO.Buffer(0 .. 15) := (others => 0);
   Channel : constant NM.DMA.DMA_Channel_Id := 0;
   -- Wait for a buffer with important numbers
   for I in NM.GPIO.Buffer'Range
      Buffer (I) := NM.GPIO.I (I mod 255);
   end loop;
   Port.Enable (Tx => True);
   Port.Configure (Config => Inherit => nil);

   NM.DMA.Enable;
   NM.DMA.Configure
      Channel => Channel,
      Config => (
         Chain_To => Transfer_A,
         Ring_Size => Ring_None,
         Increment_Write => False,
         Trigger => SPI0_TX,
         Size => Byte,
         Enable => True),
   Channel => Channel,
   From => Buf'Address,
   To => Port.Data_Register,
   Count => NM.GPIO.Buffer'Length);

   -- Do some other work while the transfer is in progress

   if not NM.DMA.Busy (Channel) Then
      -- Transfer completed, maybe begin another one.
   end if;
   NM.DMA.Abort (Channel);
end DMA_Demo;
```

The **DMA_Interrupt** procedure second channels many other variants such as reading/writing big buffers, chaining another DMA channel after a transfer finished, and generating checksums of data transferred through a DMA channel. See the RP2040 datasheet for more information on these options.

Programmable I/O (PIO)

The PIO peripheral consists of several state machines connected to all of the RP2040's GPIO pins. The PIO has its own opcodes and assembly language documented in the RP2040 datasheet. The pioasm assembler can output assembled code as a text file with the corresponding Pinout definitions.

```
pioasm -o hello -n hello hello.pio generate.txt.pio
```

The RP2040 library is a wrapper for the output of the pioasm. It's also a more available example code and documentation for PIO use that is necessary. The pio_blink example demonstrates loading an assembled PIO program and running it.

A more involved example is the RP2040-Playing_Pong, implementing simple Pong using two pioasmm programs and the Pinouts.

The Programmable I/O (PIO) chapter of the RP2040 datasheet provides an overview of the PIO hardware.

The hightemp example uses the **RP.PIO.Set_Pwm** to subset.

```
declare
   use RP.PIO;
   Pin : RP.GPIO.Pin_Pwm_Point;
   Set_Samples_Done : Ok := False;
   Buf : RP.PIO.Buffer (0 .. 511);
   -- This should be better here, use the adc_continuous example.
   loop
      Sample := RP.PIO.Read;
   end loop;
end;
```

Samples will be dropped if **Read** isn't called fast enough. In most cases, you will want to setup a DMA channel triggered on the ADC to copy samples to a buffer while the CPU does processing. The adc_continuous example demonstrates using DMA and double buffering to capture audio without missing samples.

Direct Memory Access (DMA)

The RP2040 supports multi-channel DMA. Memory location to another, freeing up the CPU to do other work while a transfer is in progress. A DMA channel is configured and started using the **System.Address** and can be triggered by almost every other peripheral. The DMA peripheral can also trigger itself to transfer the next block in a chain. Interrupts are always enabled, some address.

For example, if you had a large buffer that needed to be transmitted on SPI, you could set the Source address to the beginning of the buffer and the target address to the SPI FIFO address, with the DMA configured to increment the source address until it reaches the end of the buffer.

```
with NM, GPIO;
with Ada.Text_IO;
with NM.DMA, nvm NM.DMA;
with NM.Devices;
with NM.GPIO;

procedure DMA_Demo is
   Port : NM.GPIO.GPIO_Port renames NM.Device.SPI_B;
   Buf : NM.GPIO.Buffer(0 .. 15) := (others => 0);
   Channel : constant NM.DMA.DMA_Channel_Id := 0;
   -- Wait for a buffer with important numbers
   for I in NM.GPIO.Buffer'Range
      Buffer (I) := NM.GPIO.I (I mod 255);
   end loop;
   Port.Enable (Tx => True);
   Port.Configure (Config => Inherit => nil);

   NM.DMA.Enable;
   NM.DMA.Configure
      Channel => Channel,
      Config => (
         Chain_To => Transfer_A,
         Ring_Size => Ring_None,
         Increment_Write => False,
         Trigger => SPI0_TX,
         Size => Byte,
         Enable => True),
   Channel => Channel,
   From => Buf'Address,
   To => Port.Data_Register,
   Count => NM.GPIO.Buffer'Length);

   -- Do some other work while the transfer is in progress

   if not NM.DMA.Busy (Channel) Then
      -- Transfer completed, maybe begin another one.
   end if;
   NM.DMA.Abort (Channel);
end DMA_Demo;
```

The **DMA_Interrupt** procedure second channels many other variants such as reading/writing big buffers, chaining another DMA channel after a transfer finished, and generating checksums of data transferred through a DMA channel. See the RP2040 datasheet for more information on these options.

Programmable I/O (PIO)

The PIO peripheral consists of several state machines connected to all of the RP2040's GPIO pins. The PIO has its own opcodes and assembly language documented in the RP2040 datasheet. The pioasm assembler can output assembled code as a text file with the corresponding Pinout definitions.

```
pioasm -o hello -n hello hello.pio generate.txt.pio
```

The RP2040 library is a wrapper for the output of the pioasm. It's also a more available example code and documentation for PIO use that is necessary. The pio_blink example demonstrates loading an assembled PIO program and running it.

A more involved example is the RP2040-Playing_Pong, implementing simple Pong using two pioasmm programs and the Pinouts.

The Programmable I/O (PIO) chapter of the RP2040 datasheet provides an overview of the PIO hardware.

Interpolator

Figure 5. Interpolator Block Diagram

The Interpolator is capable of performing an Add, Right Shift, Mask, and Sign Extend all within a single CPU cycle, with feedback from previous results. The RP2040 datasheet describes the Interpolator and Sign Extend all within a single CPU cycle, with feedback from previous results. The RP2040 datasheet describes the Interpolator operations.

The RP.Interpolator package directly exposes the registers of the Interpolator peripheral. We don't have an example for this yet because we don't entirely understand it. You could write one!

ROM functions

The RP2040's ROM contains a library of useful functions that can be accessed via a symbol lookup table. The RP.ROM package provides an Ada binding to this library.

Floating point library

The RP2040's ROM includes a set of floating point functions using the IOREQ algorithm, implemented in assembly. The Cortex-M0+ does not have hardware floating point support. The RP.Float package provides a wrapper around the ROM functions. Operations are connected to **Ada.Numerics.Elementary_Functions**. Single precision floating point operations are accelerated.

The trigonometric functions (sin, tan, cos, etc) only accept a the argument range from zero to single cycle. (1.0f0 .. 1.0f0) so these are not used for acceleration by default. You can call these functions directly if your application can accept these limitations. Set pico-sdk-rp2040-fast-trig=true to use the fast versions of the functions.

```
A : constant Float := RP.ROM.Floating_Point.Fsin (10.0);
```

Multicore

The second CPU core is in the RP2040's sub-1 to delay the startup code. To make it up and begin executing code, the RP.Multicore package provides a **Launch_Core1** procedure.

See the multicore example for usage instructions.

Custom board support

The focus of this project is to support the RP2040 on the Raspberry Pi Pico development board, but these libraries are portable to any board that uses the RP2040. The recommended changes to the **pseudoboot/boot_map** files contain the pulsed-mode pins, pin-multiplexer, and peripheral devices for your custom board's components.

Boot code

The RP2040 ROM has no onboard flash, even to connected an external flash chip. The **RP2_XIP2** Executable Flash Image Generator (RP2_XIP2) generates a binary image that can be written to the RP2040's flash memory. The RP2040 ROM expects the first 256 bytes to read commands to initialize the external flash chip. The RP2040 ROM, which is a subset of the Pico-sdk, then jumps to the first begins executing. This section is known as **boot2** in linker scripts, which should disable the XIP, reconfigure it for the specific flash chip on the board, then jump to the main program. At the beginning of user code, now executing from the memory-mapped flash chip.

GCC/ada_rts includes a boot2 section in its linker scripts. As the compiled boot2 needs to be patched to 256 bytes with a checksum enabled, the GCC/ada_rts repository contains pre-compiled boot2 code that will be linked automatically.

The RDP uses should the **Flash_Chip** condition **alias.** boot2 to select the correct boot2 for your board.

```
[configuration.values]
GCC/ada_rts.flash_chip = "w25q64"
```

The following boot2 variants are supported:

generic_03

The generic_03 boot2 is compatible with any flash chip that supports the 03h read command (which is most serial NOR flash devices). The downside is that it operates at a reduced speed.

generic_quad

The generic_quad boot2 is compatible with most (but not all) QSPI NOR flash chips. It is compatible with at least the following flash chips:

- W25Q080
- W25Q16JV
- S25FL064P
- GD25Q16C
- W25Q32JV
- W25Q64JV
- W25Q128JV
- W25Q256JV

W25Q080

This boot2 is compatible with Winbond devices that have the QE flag in bit 6 of Status Register 2, and support writing to Status Register 2 without Status Register 1. It is compatible with at least the following Winbond devices:

- W25Q80V
- W25Q16JV
- W25Q32JV
- W25Q64JV
- W25Q128JV
- W25Q256JV

W25Q16JV

This boot2 is compatible with Winbond devices that have the QE flag in bit 6 of Status Register 2, and support writing to Status Register 2 without Status Register 1. It is compatible with at least the following Winbond devices:

- W25Q80V
- W25Q16JV
- W25Q32JV
- W25Q64JV
- W25Q128JV
- W25Q256JV

Linker script

Some startup code and a linker script are needed to generate an executable binary for the RP2040. If you are using a CPP runtime, libNM_RP.gnat provides reasonable defaults in the **bsp/** startup directory. If you wish to customize these, copy both directories to your RDP crate and set the following in your project.

```
[configuration.values]
libNM_RP.bsp_dir = "bsp/" + Project.Dir
```

Pin definitions

Pins setting that your custom board has the same pinout as the Pico. Most RDP crates will include a package that defines symbolic names for GPIO pins that match a schematic let's name a reference chart a user of your board would recognize.

```
[configuration.values]
libNM_RP.pin_dir = "pins/" + Project.Dir
```

HSE startup delay

Some boards, like the Adafruit Feather RP2040, have higher capacitance on the crystal oscillator traces and need to wait a longer **HSE_Startup_Delay** to initialize.

```
RF.Clock.Initialize
   (XOSC_Frequency => 12_000_000,
    HSE_Startup_Delay => 50_000);
-- Initialize the XOSC with a 12 MHz crystal and a 50 ms startup delay.
```

Optimization

Binary size

The RP2040's XIP cache is only 16 KB and reads from flash RDP are fairly slow compared to CPU cycle time and memory access. The **RP.Flash.Ram_Execute** procedure can be used to temporarily disable the cache and measure its performance impact on your program.

The RP.Devices.Initialize function enables every driver and device on the chip during initialization. This is a convenient for the time to includes the code for every driver it elaboration in your binary, even if they're not used. This adds nearly 15 KB of code to the binary and slows the boot time. If you don't need every driver, you don't want to reduce binary size, you can copy the RP.Devices.Initialize and remove the unused drivers.

Switches

While compile time optimizations are no substitute for efficient algorithms, GCC can usually make your code fast and small. Optimization can make debugging difficult, so it's best to start with a debug build and only turn on optimizations after the code is bug-free. **Switches** options The GNAT user guide includes a list of optimization options. The example below shows the ones I use most frequently.

```
package Compiler
   for Default_Switches ("Ada") use (
      "-O2",  -- Optimize for size, O2 -O2 -O3 for performance
      "-gnatp",
      "-g",   -- Generate debug symbols
      "-gnatn",  -- Allow inlining
      "-gnatwa",  -- Enable all warnings
      "-gnatVa",  -- Validate checks, function tester without these, but
      "-gnatwe",  -- Treat warnings as errors
      "-gnatyO3M120",  -- Line length, Consistent,
      "-gnatwl",  -- Warnings for elaboration,
      "-gnatwh",  -- Warnings for hidden declarations, PM and RM conditions are enforced,
      "-gnatwu",  -- Enable Unused, Procedures also need to be marked with
      "-gnatwn",  -- Warnings for null exclusions,
      "-gnatwz",  -- Warnings for zero-length arrays,
      "-gnatwb",  -- Warnings for bad fixes,
      "-gnatwf",  -- Warnings for formal mode,
      "-gnatwt",  -- Warnings for tag mismatches,
      "-gnatwc",  -- Warnings for conditionals,
      "-gnatwd",  -- Warnings for dispatching,
      "-gnatwp",  -- Warnings for pragmas,
      "-gnatwr",  -- Warnings for representation clauses,
      "-gnatwo",  -- Warnings for overlays,
      "-gnatwm",  -- Warnings for modes,
      "-gnatwi",  -- Warnings for implementation,
      "-gnatwk",  -- Warnings for constants,
      "-gnatwv",  -- Warnings for volatile,
      "-gnatwx",  -- Warnings for unused formals,
      "-gnatwy",  -- Warnings for obsolescence
```

```
                    Error_Busy,
                    Error_Overrun,
                    Error_Parity,
                    Error_Protocol,
                    Error_Hardware_Overrun,
                    Error_Timeout,
                    Error_Hardware_Error,
                    Error_Unknown,
                    Error_Overflow,
                    Error_Bad_Port_Num,
                    Error_Value_Out_Of_Range,
                    Error_Not_Supported,
                    Error_Not_Ready,
                    Error_Not_Open,
                    Error_Already_Open,
                    Error_Not_Initialized,
                    Error_Bad_Parameter,
                    Error_Bad_Handle,
                    Error_Access_Denied,
                    Error_Device_Not_Found,
                    Error_Not_Enough_Memory,
                    Error_Internal_Error,
                    Error_Hardware_Failure,
                    Error_Data_Error,
                    Error_Hardware_Malfunction,
                    Error_Timeout,
                    Error_Hardware_Stalled,
                    Error_Device_Not_Responding,
                    Error_Device_Not_Configured,
                    Error_Unexpected_Network_Change,
                    Error_Connection_Refused,
                    Error_Other,
                    Error_Not_Enough_Bandwidth,
                    Error_Interrupted,
                    Error_Interrupted_System_Call,
                    Error_Unsupported_Baud_Rate,
                    Error_Unsupported_Data_Size,
                    Error_Unsupported_Stop_Bits,
                    Error_Unsupported_Parity,
                    Error_Unsupported_Handshaking,
                    Error_Unsupported_Operation,
                    Error_Buffer_Too_Small,
                    Error_Buffer_Too_Large,
                    Error_Invalid_Buffer,
                    Error_Invalid_Length,
                    Error_Invalid_Offset,
                    Error_Invalid_Data,
                    Error_Invalid_Stop_Bits,
                    Error_Invalid_Parity,
                    Error_Invalid_Handshaking,
                    Error_Invalid_Operation,
                    Error_Invalid_Baud_Rate,
                    Error_Invalid_Data_Size,
                    Error_Invalid_Port_Num,
                    Error_Invalid_Handle,
                    Error_Invalid_Parameter,
                    Error_Invalid_Pointer,
                    Error_Invalid_Version,
                    Error_Invalid_Flags,
                    Error_Invalid_Reserved,
                    Error_Invalid_Enum,
                    Error_Invalid_Size,
                    Error_Invalid_Alignment,
                    Error_Invalid_Address,
                    Error_Invalid_Type,
                    Error_Invalid_Function,
                    Error_Invalid_Index,
                    Error_Invalid_Value,
                    Error_Invalid_Key,
                    Error_Invalid_Name,
                    Error_Invalid_Path,
                    Error_Invalid_Mode,
                    Error_Invalid_Flag,
                    Error_Invalid_Reserved_Flag,
                    Error_Invalid_Enum_Value,
                    Error_Invalid_Size_Value,
                    Error_Invalid_Alignment_Value,
                    Error_Invalid_Address_Value,
                    Error_Invalid_Type_Value,
                    Error_Invalid_Function_Value,
                    Error_Invalid_Index_Value,
                    Error_Invalid_Value_Value,
                    Error_Invalid_Key_Value,
                    Error_Invalid_Name_Value,
                    Error_Invalid_Path_Value,
                    Error_Invalid_Mode_Value,
                    Error_Invalid_Flag_Value,
                    Error_Invalid_Reserved_Flag_Value,
                    Error_Invalid_Enum_Value_Value,
                    Error_Invalid_Size_Value_Value,
                    Error_Invalid_Alignment_Value_Value,
                    Error_Invalid_Address_Value_Value,
                    Error_Invalid_Type_Value_Value,
                    Error_Invalid_Function_Value_Value,
                    Error_Invalid_Index_Value_Value,
                    Error_Invalid_Value_Value_Value,
                    Error_Invalid_Key_Value_Value,
                    Error_Invalid_Name_Value_Value,
                    Error_Invalid_Path_Value_Value,
                    Error_Invalid_Mode_Value_Value,
                    Error_Invalid_Flag_Value_Value,
                    Error_Invalid_Reserved_Flag_Value_Value,
                    Error_Invalid_Enum_Value_Value_Value,
                    Error_Invalid_Size_Value_Value_Value,
                    Error_Invalid_Alignment_Value_Value_Value,
                    Error_Invalid_Address_Value_Value_Value,
                    Error_Invalid_Type_Value_Value_Value,
                    Error_Invalid_Function_Value_Value_Value,
                    Error_Invalid_Index_Value_Value_Value,
                    Error_Invalid_Value_Value_Value_Value,
                    Error_Invalid_Key_Value_Value_Value,
                    Error_Invalid_Name_Value_Value_Value,
                    Error_Invalid_Path_Value_Value_Value,
                    Error_Invalid_Mode_Value_Value_Value,
                    Error_Invalid_Flag_Value_Value_Value,
                    Error_Invalid_Reserved_Flag_Value_Value_Value,
                    Error_Invalid_Enum_Value_Value_Value_Value,
                    Error_Invalid_Size_Value_Value_Value_Value,
                    Error_Invalid_Alignment_Value_Value_Value_Value,
                    Error_Invalid_Address_Value_Value_Value_Value,
                    Error_Invalid_Type_Value_Value_Value_Value,
                    Error_Invalid_Function_Value_Value_Value_Value,
                    Error_Invalid_Index_Value_Value_Value_Value,
                    Error_Invalid_Value_Value_Value_Value_Value,
                    Error_Invalid_Key_Value_Value_Value_Value,
                    Error_Invalid_Name_Value_Value_Value_Value,
                    Error_Invalid_Path_Value_Value_Value_Value,
                    Error_Invalid_Mode_Value_Value_Value_Value,
                    Error_Invalid_Flag_Value_Value_Value_Value,
                    Error_Invalid_Reserved_Flag_Value_Value_Value_Value,
                    Error_Invalid_Enum_Value_Value_Value_Value_Value,
                    Error_Invalid_Size_Value_Value_Value_Value_Value,
                    Error_Invalid_Alignment_Value_Value_Value_Value_Value,
                    Error_Invalid_Address_Value_Value_Value_Value_Value,
                    Error_Invalid_Type_Value_Value_Value_Value_Value,
                    Error_Invalid_Function_Value_Value_Value_Value_Value,
                    Error_Invalid_Index_Value_Value_Value_Value_Value,
                    Error_Invalid_Value_Value_Value_Value_Value,
                    Error_Invalid_Key_Value_Value_Value_Value_Value,
                    Error_Invalid_Name_Value_Value_Value_Value_Value,
                    Error_Invalid_Path_Value_Value_Value_Value_Value,
                    Error_Invalid_Mode_Value_Value_Value_Value_Value,
                    Error_Invalid_Flag_Value_Value_Value_Value_Value,
                    Error_Invalid_Reserved_Flag_Value_Value_Value_Value_Value,
                    Error_Invalid_Enum_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Size_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Alignment_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Address_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Type_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Function_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Index_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Key_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Name_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Path_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Mode_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Flag_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Reserved_Flag_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Enum_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Size_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Alignment_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Address_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Type_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Function_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Index_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Key_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Name_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Path_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Mode_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Flag_Value_Value_Value_Value_Value_Value_Value,
                    Error_Invalid_Reserved_Flag_Value
```

RT_SysTick_Init_Delay() procedure would hang indefinitely.

The SysTick interrupt handler for SysTick IRQ (SysTick_Handler) was called, when it tried to do this, the PendSV interrupt was also triggered, which led to a crash because this interrupt is not defined. PendSV is meant to be used by an RTOS to implement preemptive task switching. Since it was not implemented, the system crashed when the interrupt was triggered.

I've chosen to remove RT_SysTick_Init() from u220_hal_rather than fix the interrupt for the following reasons:

- SysTick is Cortex-M0 specific hardware, not a generic timer. Mixing it into a generic HAL makes it harder to port, which we want to support.
- Most ARM Cortex-M compatible RTOS libraries will configure SysTick and PendSV on their own, conflicting with u220_hal's implementation.
- SysTick doesn't do anything that we can't already do with RT Timer.

RT_Interrupts implements weak handlers for all user interrupts

This allows **__weak** symbols to be overridden in user code. Previously, the default handler for these interrupts would call the **Hal** function, causing a linking error or crash if no handler was defined.

RT_Interrupts_Initialize() and RT_Interrupts_Deinitialize()

RT_Interrupts_Initialize() and RT_Interrupts_Deinitialize(). This function, **RT_Interrupts_Initialize()** is called on any of **process_procedure** specified by calls to **RT_Interrupts_Attach_Handler()**. If no handler is defined for an interrupt, the **Program_Error** exception is triggered, indicating a missing handler implementation. These packages work with CPP interrupts.

If you need to define a custom interrupt, you can use **RT_Interrupts_Attach_Handler** or **RT_Interrupts_Attach** in your app. The **u_hal_systick** contains the **Enhanced_Mode** option. See the **u_interrupt_example**.

1.4.1

April 4, 2022

Bug Fixed

RDP mode symbols were overridden by the toolchain

The **STM32_**__**mode** symbols were weak in order to allow users to override them, but this caused the toolchain's symbols to take precedence. The change to make these symbols weak has been reverted with this release.

1.4.0

April 3, 2022

New Features

Multicore support

The RP Multicore, RP Multicore Spinlocks, and RP Multicore FIFO packages expose the second CPU core and the inter-core communication hardware. Multicore support is enabled by the new **Enable_Multicore** option in u220_Thermacle_Hardware operation was supported by the Raspberry Pi pico/pico-w boards. These packages work with CPP interrupts.

Note that many of the drivers in u220_hal are not safe for concurrent access and debugging over connections between cores can be difficult. If you are considering, or forking anyone is considering to use core at a time, or guard accesses with the spinlocks.

Thanks to @Fabien-Chouteau for contributing the multicore driver.

UART and SPI Interrupts

The RP UART and RP SPI drivers now have procedures for enabling and seeding operational interrupt flags. These interrupts may be triggered from non-interrupt code. The general-purpose interrupt handler, Thanks to @Fabien-Chouteau for these changes.

Flash cache control

Normally, all reads from the external QSPI flash use a read through cache. The RP Flash Cache package can disable and flush the cache. Cache access and XIP counters can provide information on cache performance. Disabling the cache may be useful if you are concerned about the effect of cache misses on real-time performance. Disabling the cache may also be useful for debugging. **Enable_RP_Flash_Cache_Power_Down** procedures may reduce power consumption when the cache is disabled.

RDP initialization with Revocation

The **u_rp_init_initialize** option has been renamed to **__rp_initialize_bootsel**. Its better compatibility with Raspberry's official tools.

Bug Fixed

RP2350.RESET2 reset for PSU

When **RP_PSU.RESET_Initialize** was called, it would reset the entire PSU peripheral, meaning this driver could not be used concurrently with other PSU drivers.

RP2350.RESET2 RESET order

RAM.Core values were reinstated correctly when the **Init_RAM** procedure was used.

1.3.0

February 28, 2022

New Features

DMA Setup can be performed without triggering a transfer

The **Enable_Transfer** option in the DMA Setup procedure can be used to address of a channel without triggering it immediately. Previously, RP DMA Setup both set up the channel and triggered a transfer. This change allows the configuration and RP DMA Start with only a Channel argument will trigger the transfer. (P from. To, and Count are passed as arguments to Start. Here the old behavior is retained.)

Note that if Enhanced_Read or Enhanced_Write are True, multiple transfers will be triggered. RP DMA Start will wait for the first and for subsequent transfers to be triggered by the DREQ signal. Setting Enhanced_Read and Enhanced_Write Dreq to zero will trigger the DMA immediately.

SysTick Delay_Until

The SysTick driver now has a Delay_Until procedure with functionality similar to **RT_Time.Delay_Until**. All SysTick delays are tested to be accurate to 1 tick.

Flash programming

The RP Flash driver now erases and program the flash chip connected to the RP2040's XIP interfaces, which is where code executes from. Note that these must be called before the **Enable_Flash_Cache** procedure.

Thanks to Fabien Chouteau for contributing this driver.

ITO instruction encoding

The RP PIO driver now has a procedure to encode instructions for each PIO opcode. The procedure's function return **PIO_Instruction** and can be used to fill **RP_PIO_Program** arrays. See the **pio_example** example.

RISCOVER and Radio_CSR driver

A PIO program that implements the wire protocol for RISCOVER RISC-V DTM has been added as a child package of RP PIO. The Radio_CSR driver was added to the RP PIO package.

Tests

Tests for RP PIO, RP I2C and Flash functions were added. GitHub runners now supported for tests and reports CSV information coverage for u220_hal.

Bug Fixed

I2C Mem_Write sent a repeated start after the address

The RP I2C driver's Mem_Write procedure was supposed to send a repeated start between the memory address and the data to be written. This caused issues for some I2C EEPROMs. Now, the driver sends the memory address and data both in one Master_Transmit call.

Thanks to Roger Rodgers for reporting the bug.

SysTick interrupt incorrect values

RT_SysTick.Clock was returning the value of the **CURRENT** register, not the ticks counted by the Time Interrupt handler. **RT_SysTick.Clock** now returning the value of the **CURRENT** register, which is the number of milliseconds since SysTick was enabled.

RP.RTC Get_Clock incorrect values

Get_Clock could throw a Constraint_Error if the hardware RTC year was 0.

RP2040.Pending_Point used 32-bit reads

The Pending_Point procedure did not check the RDY register before reading functions that are only available in V2. These calls could hang.

- initialize
- uninitialize
- freeRunning
- freeRunning0
- freeRunning1

Invalid Scenes clarification

The Scenes procedure's error codes table that included a Difficult with overview exception Scenes in a common header. These The error codes table that included a Difficult with overview exception Scenes in a common header. These The error codes table that included a Difficult with overview exception Scenes in a common header. These The

PWM dividers can have a value of zero

The documentation is unclear on what this means, but my testing shows that it acts like a divider of 1, which outputs the clk_sys frequency.

Fast I2C writes would result in dropped bytes

The RP.I2C_Master driver has been modified to wait for the TX FIFO to be empty before writing a byte. This effectively reduces the FIFO depth to 1 byte. This is the same behavior as the upstream SDK.

Known issues

I2C clock is slower than expected

In 400 KHz (fast mode) operation, the I2C master generates SCL at approximately 380 KHz. I believe this is due to clock stretching caused by the new TX FIFO blocking behavior. The upstream SDK has the same behavior. According to the I2C specification, a fast mode clock may be **up to** 400 KHz, but specifies no minimum frequency. It may be possible to workaround this by using DMA to write to the I2C FIFO, but this is untested.